

Weekly Progress Report

Cloud Based Research Paper Summarization Tool

NYU Cloud Computing • Spring 2026

Members: Shreyas Sankpal, Pranjal Mishra, Yang Zheng, Kerry Huang

Project Overview

The goal of our project is to make it easier to go through academic research papers without having to read every word. Users can search for papers from arXiv and Semantic Scholar, and the tool pulls the full text and produces a structured summary using an LLM running on Amazon Bedrock. On top of that, we built a chat interface so users can ask questions about a specific paper, a knowledge graph showing key concepts and how they relate, and a way to find similar papers from your own history. The whole thing runs serverlessly on AWS, so there is no cost when nothing is happening.

Week 1 - April 6, 2026

Progress Summary

We spent most of week one getting everyone on the same page. The four of us went through the project requirements together, split up ownership areas, and started looking into what tech stack made the most sense. Nothing was built yet, but we left the week with a clear idea of who is responsible for what and a shortlist of tools to evaluate.

Activities Completed

- **Project Kickoff:** Went through the requirements doc as a group and agreed on the project scope.
- **Technology Research:** Compared AWS, GCP, and Azure for serverless support. Decided to go with AWS.
- **Role Assignment:** Pranjal takes Architecture and Frontend, Yang takes Search, Shreyas takes Pipeline and Orchestration, Kerry takes Auth and API.
- **API Research:** Looked into the arXiv and Semantic Scholar public APIs to check what data is available and how the rate limits work.

Next Steps

- Put together a proper architecture diagram and finalize the AWS service choices.
- Compare LLM options: Bedrock, OpenAI, and Anthropic direct.
- Write up the system architecture doc so we have something concrete to review in Week 2.

Week 2 - April 13, 2026

Progress Summary

This week we finalized the architecture and got the official diagram done. We went fully serverless on AWS, which should keep costs low and scale without us having to manage any servers. The diagram is done and we are using it as the reference going into the build phase.

Cloud System Architecture

Here is a breakdown of what each piece of the AWS setup is doing:

- **Frontend Hosting:** Static site sits in an S3 bucket and gets served through CloudFront.
- **Authentication:** Cognito handles user sign-up, login, email verification, and the forgot-password flow.
- **API Routing:** API Gateway sits in front of everything and routes requests to the right Lambda.
- **Compute Layer:** All the business logic lives in Lambda functions (Node.js 20). We defined six CDK stacks: Auth, Data, Pipeline, Api, Frontend, and Ops.
- **Database:** DynamoDB with a single-table design and a GSI for storing user data, sessions, and summaries.
- **File Storage:** A second S3 bucket holds the raw PDFs we download. Files get cleaned up after 90 days.
- **Async Processing:** SQS with a Dead Letter Queue handles document ingestion and chunking in the background so the frontend does not time out on large PDFs.
- **AI Layer:** We picked Amazon Bedrock for the LLM side, using Qwen 3 Next 80B for text generation (with a Claude Sonnet swap ready to go) and Titan Text Embeddings v2 for the vector work.

AWS Service	Architecture Layer	Purpose in Project
Amazon S3	Frontend / Presentation	Hosts the static web interface (CloudFront origin).
Amazon Cognito	Authentication	User registration, login, session management.
Amazon API Gateway	Network / Routing	Routes all frontend requests to backend Lambda handlers.
AWS Lambda	Compute / Backend	Executes business logic, DB interactions, and search integrations.
Amazon DynamoDB	Database	Single-table store for users, sessions, and summaries.
Amazon SQS + DLQ	Async Processing	Queue for PDF ingestion/chunking; DLQ captures failures.
Amazon S3 (PDF)	File Storage	Holds downloaded raw PDFs; 90-day lifecycle rule.
Amazon Bedrock	AI / LLM	Qwen 3 summarization + Titan embeddings for RAG.

AWS Step Functions	Orchestration	State machine: Fetch > Extract > Chunk > MapSummarize > ReduceSummary.
Amazon CloudFront	CDN	Distributes the frontend globally with path-rewriter Function.
CloudWatch / X-Ray	Observability	Dashboard, 4 alarms to SNS, distributed tracing on every Lambda.

Next Steps

- Start building. Get the AWS environment set up and connect the frontend to API Gateway.
- Scaffold the Next.js frontend and wire up the Cognito auth flow.
- Get at least one Lambda running end-to-end.

Week 3 - April 20, 2026

Progress Summary

Week 3 is when we started actually building. Pranjal got the CDK bootstrapped and the first two stacks deployed, Kerry had Cognito auth working end-to-end including JWT injection into API calls, and Yang had a working arXiv search returning live results. By end of week a user could sign up, log in, type a search query, and see real papers come back in the browser. Shreyas also got the first pipeline Lambda up, which handles downloading PDFs to S3.

Activities Completed

- **CDK Bootstrap and Stack Deployment (Pranjal):** Auth and Data stacks deployed. Cognito User Pool, DynamoDB table, and PDF bucket are live in AWS.
- **Frontend Scaffolding (Pranjal):** Next.js 14 project set up with Tailwind. Pages defined for /, /search, /summary/[id], and /chat/[id].
- **Cognito Auth Flow (Kerry):** Email verification, SRP login, forgot-password, and JWT injection all working and tested.
- **API Gateway and Lambda Skeleton (Kerry):** API stack deployed with Cognito JWT authorizer. Stub handlers for /search, /submit-job, /get-summary, /list-summaries, and /get-quota are live.
- **arXiv Search Integration (Yang):** arXiv client written in TypeScript. The Search Lambda hits the arXiv API and deduplicates results by arxivId. Integration tests added.
- **PDF Fetch Lambda (Shreyas):** First pipeline Lambda (fetch-pdf) is working. Downloads a PDF from arXiv to S3, with error handling for papers that are not publicly available.

Issues We Ran Into

- **Lambda Cold Starts:** First Lambda calls were taking around 1.8 seconds. Enabling SnapStart and trimming the bundle size brought this down to an acceptable level.
- **CORS on API Gateway:** Preflight requests were getting blocked when running the frontend locally. Fixed by enabling CORS on all resources and making sure the Lambda handlers returned the right headers.

Key Metrics This Week

Metric	Value
CDK Stacks deployed	2 of 6 (Auth, Data)
Lambda functions live	3 (search, submit-job stub, fetch-pdf)
arXiv search results (sample query)	~20 deduplicated papers returned
Vitest integration tests written	4
Frontend routes scaffolded	4

Next Steps

- Build out the full Step Functions summarization pipeline (Extract, Chunk, MapSummarize, ReduceSummary).
- Add Semantic Scholar to the search flow and merge results with arXiv output.

- Hook up the frontend search page to the live Lambda so results come from the real API.
- Start on the RAG chat backend, which needs chunk embeddings stored in DynamoDB for similarity lookup.

Week 4 - April 27, 2026

Progress Summary

Week 4 is when everything came together. The full summarization pipeline is live, the chat and knowledge graph features are working, all six CDK stacks are deployed, and the app is running in production. A paper goes from search to a full five-section summary in about 30 seconds and costs roughly \$0.20 in Bedrock tokens. When nothing is being processed, the cost is \$0.

Activities Completed

- **Step Functions Pipeline (Shreyas):** Full state machine is deployed and working. Steps: Fetch, Extract, Chunk, MapSummarize, ReduceSummary. Produces structured summaries with five sections: Objectives, Methodology, Results, Limitations, and Contributions.
- **Semantic Scholar Integration (Yang):** Semantic Scholar client is done. Both arXiv and Semantic Scholar are queried in parallel and results are deduplicated by arxivId, DOI, or normalized title. All 14 integration tests are passing.
- **RAG Chat (Pranjal):** Talk-to-PDF feature is working. Chunk embeddings using Titan Text Embeddings v2 are stored in DynamoDB. When a user asks a question, the most relevant chunks come back with inline citation tooltips.
- **Knowledge Graph (Pranjal):** The ReduceSummary Lambda now also extracts typed entities and relationships. These get rendered in a React Flow graph with a key-concepts panel on the side and a regenerate button.
- **Related Papers (Pranjal):** The top 3 similar papers from the user's history are shown using cosine similarity over mean paper embeddings.
- **Export and Dedup (Pranjal):** Users can copy a summary as Markdown, download it as .md, or get a BibTeX file. If a paper has already been summarized by any user, the cached result is returned so we do not re-run Bedrock on the same content.
- **Ops Stack (Pranjal):** CloudWatch dashboard is live with four alarms going to SNS email. X-Ray tracing is on every Lambda. AWS Budgets alarm is set at \$50/month.
- **Frontend (Pranjal):** Static export finalized and deployed to S3 behind CloudFront. All pages including search, summary, chat, and graph are working and responsive.
- **CI/CD (All):** GitHub Actions runs lint and build on every pull request. The deploy.sh script handles bootstrapping, deploying all six stacks, building the frontend with the right Cognito and API config, uploading to S3, and invalidating the CloudFront cache.

Final Architecture Summary

Stack	Key Resources
Auth	Cognito User Pool + SPA client
Data	DynamoDB single-table + GSI1 + S3 PDFs bucket (90-day lifecycle)
Pipeline	SQS + DLQ + Step Functions state machine + 6 Lambdas

Api	API Gateway REST + Cognito JWT authorizer + 8 handler Lambdas
Frontend	Private S3 + CloudFront with path-rewriter Function
Ops	SNS + 4 CloudWatch alarms + Budgets alarm + Dashboard

Final Metrics

Metric	Value
End-to-end summary time	~30 seconds
Cost per paper	~\$0.20 (Bedrock tokens)
Idle infrastructure cost	\$0
Total Lambda functions	14 (8 API + 6 pipeline)
CDK stacks deployed	6 of 6
Vitest integration tests	14 (all passing)
Live demo URL	https://d24irdkbe9jj2b.cloudfront.net
AWS Budgets alarm threshold	\$50/month

Lessons Learned

- **Lambda cold starts are a real UX problem:** We did not think much about this early on, but 1.8 second delays were noticeable. SnapStart helped a lot and is worth enabling from the start.
- **Step Functions was the right call over chaining Lambdas manually:** The visual debugger alone saved us hours. Retry logic and error handling come built-in, and being able to see the state machine in the AWS console made it easy to spot where things were failing.
- **Content-hash dedup keeps costs manageable during testing:** We were re-summarizing the same papers over and over while debugging. Hashing on arxivId plus version and skipping Bedrock on cache hits made a noticeable difference in development costs.
- **TypeScript CDK catches issues before deploy:** Several IAM policy mistakes and missing environment variables got caught at synth time. Would have been a lot more painful to debug after deploying.

Project Status

Status: v1.0 is live. All planned features are shipped. Final report, architecture diagrams, and execution guide are all done and checked in.